Name :- Divya Ajay Pandey
Prn:- 202501050016

2.1. Practice Lab Assignments
2.1.1. List Operations
2.1.2. Dictionary Operations
2.2. Lab Assignments
2.2.1. Linear Search Technique
2.2.2. Captain of the Team

Practical 02
2.1. Practice Lab Assignments
2.1.1. List Operations
Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:
1. Add
2. Remove
3. Display
4. Quit
The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:
● Add: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
● Remove: Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
● Display: Displays the current list of integers. If the list is empty, display "List is empty".
● Quit: Exits the program.
● The program should handle invalid menu choices by displaying "Invalid choice".Ensure that the program continues to prompt the user until they choose to quit (option 4).
Test Case 1
1.·Add⏎
2.·Remove⏎
3.·Display⏎
4.·Quit⏎
Enter·choice:·1
Integer:·11
List·after·adding:·[
11]⏎
1.·Add⏎
2.·Remove⏎
3.·Display⏎
4.·Quit⏎

```
Enter·choice:·1
Integer:·25
List·after·adding:·[
11,·25]⏎
1.·Add⏎
2.·Remove⏎
3.·Display⏎
4.·Quit⏎
Enter·choice:·2
Integer:·26
Element·not·found⏎
1.·Add⏎
2.·Remove⏎
3.·Display⏎
4.·Quit⏎
Enter·choice:·2
Integer:·11
List·after·removing:
·[25]⏎
1.·Add⏎
2.·Remove⏎
3.·Display⏎
4.·Quit⏎
Enter·choice:·2
Integer:·25
List·after·removing:
·[]⏎
1.·Add⏎
2.·Remove⏎
3.·Display⏎
4.·Quit⏎
Enter·choice:·3
List·is·empty⏎
1.·Add⏎
2.·Remove⏎
3.·Display⏎
4.·Quit⏎
Enter·choice:·2
List·is·empty⏎
1.·Add⏎
2.·Remove⏎
3.·Display⏎
4.·Quit⏎
Enter·choice:·8
Invalid·choice⏎
1.·Add⏎
2.·Remove⏎
3.·Display⏎
```

4.·Quit⏎
Enter·choice:·4
```
def main():
my_list = []
while True:
# Display the Menu
print("1. Add")
print("2. Remove")
print("3. Display")
print("4. Quit")
# Prompt for entering the choice
choice = input("Enter choice: ")
if choice = "1": # add
try:
val = int(input("Integer: "))
my_list.append(val)
print(f"List after adding: {my_list}")
except ValueError:
print("Invalid input")
elif choice = "2": # Remove
# Check if list is empty before asking for input
if not my_list:
print("List is empty")
else:
try:
val = int(input("Integer: "))
if val in my_list:
my_list.remove(val)
print(f"List after removing: {my_list}")
else:
print("Element not found")
except ValueError:
print("Invalid input")
elif choice = "3": # Display
if not my_list:
print("List is empty")
else:
print(my_list)
elif choice = "4": # Quit
break
else:
print("Invalid choice")
if _name _ = " _main _":
main()
```

07:09

mitaoe.codetantra.com/

CODETANTRA

```
    print("4. Quit")

    n = int(input("Enter choice: "))

    if n == 1:
        add = int(input("Integer: "))
        l1.append(add)
        print(f"List after adding: {l1}")

    elif n == 2:
```

Test case 1

249 ms

Expected output	Actual output
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
	Enter choice:

07:10

mitaoe.codetantra.com/

CODETANTRA

```
    print("4. Quit")

    n = int(input("Enter choice: "))

    if n == 1:
        add = int(input("Integer: "))
        l1.append(add)
        print(f"List after adding: {l1}")

    elif n == 2:
```

Integer: 11	Integer: 11
List after adding: [11]	List after adding: [11]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
	Enter choice:

07:10

mitaoe.codetantra.com/

CODETANTRA

```
    print("4. Quit")

    n = int(input("Enter choice: "))

    if n == 1:
        add = int(input("Integer: "))
        l1.append(add)
        print(f"List after adding: {l1}")

    elif n == 2:
```

```
List after adding: [11, 25]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
```

```
List after adding: [11, 25]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
```

07:10

mitaoe.codetantra.com/

CODETANTRA

```
    print("4. Quit")

    n = int(input("Enter choice: "))

    if n == 1:
        add = int(input("Integer: "))
        l1.append(add)
        print(f"List after adding: {l1}")

    elif n == 2:
```

	d
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 2	Enter choice: 2
Integer: 11	Integer: 11

07:10

mitaoe.codetantra.com/

CODETANTRA

```
    print("4. Quit")

    n = int(input("Enter choice: "))

    if n == 1:
        add = int(input("Integer: "))
        l1.append(add)
        print(f"List after adding: {l1}")

    elif n == 2:
```

```
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 25
```

```
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 25
```

07:10

mitaoe.codetantra.com/

CODETANTRA

```
    print("4. Quit")

    n = int(input("Enter choice: "))

    if n == 1:
        add = int(input("Integer: "))
        l1.append(add)
        print(f"List after adding: {l1}")

    elif n == 2:
```

4. Quit	4. Quit
Enter choice: 3	Enter choice: 3
List is empty	List is empty
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display

07:10

mitaoe.codetantra.com/

CODETANTRA

```
    print("4. Quit")

    n = int(input("Enter choice: "))

    if n == 1:
        add = int(input("Integer: "))
        l1.append(add)
        print(f"List after adding: {l1}")

    elif n == 2:
```

List is empty	List is empty
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 8	Enter choice: 8
Invalid choice	Invalid choice

07:10

mitaoe.codetantra.com/

CODETANTRA

```
    print("4. Quit")

    n = int(input("Enter choice: "))

    if n == 1:
        add = int(input("Integer: "))
        l1.append(add)
        print(f"List after adding: {l1}")

    elif n == 2:
```

```
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 4
```

```
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 4
```

2.1.2. Dictionary Operations

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

● All operations must be performed using dictionary methods.
● Perform deletion only if possible; leave the dictionary unchanged.
● Refer to the visible test cases for better understanding and strictly match with the input/outputs.

```
Original·Dictionary:·{1:·'Amit',·2:·'Riya',·3:·'Kiran',·4:·'Neha',·5:·'A
rjun',·6:·'Pooja',·7:·'Rahul',·8:·'Sneha',·9:·'Vikram',·10:·'Anjali'}⏎
11
Suresh
After·Insertion:·{1:·'Amit',·2:·'Riya',·3:·'Kiran',·4:·'Neha',·5:·'Arjun
',·6:·'Pooja',·7:·'Rahul',·8:·'Sneha',·9:·'Vikram',·10:·'Anjali',·11:·'S
uresh'}⏎
3
Karthik
After·Update:·{1:·'Amit',·2:·'Riya',·3:·'Karthik',·4:·'Neha',·5:·'Arjun'
,·6:·'Pooja',·7:·'Rahul',·8:·'Sneha',·9:·'Vikram',·10:·'Anjali',·11:·'Su
resh'}⏎
5
After·Deletion:·{1:·'Amit',·2:·'Riya',·3:·'Karthik',·4:·'Neha',·6:·'Pooj
a',·7:·'Rahul',·8:·'Sneha',·9:·'Vikram',·10:·'Anjali',·11:·'Suresh'}⏎
Traversing·Dictionary:⏎
1·:·Amit⏎
```

```
2·:·Riya⏎
3·:·Karthik⏎
4·:·Neha⏎
6·:·Pooja⏎
7·:·Rahul⏎
8·:·Sneha⏎
9·:·Vikram⏎
10·:·Anjali⏎
11·:·Suresh⏎
# Initial dictionary with 10 predefined records
student = {
1: "Amit",
2: "Riya",
3: "Kiran",
4: "Neha",
5: "Arjun",
6: "Pooja",
7: "Rahul",
8: "Sneha",
9: "Vikram",
10: "Anjali"
}
def main():
# 1. Display Original Dictionary
print(f"Original Dictionary: {student}")
# 2. Insertion Operation
try:
insert_key = int(input())
insert_value = input()
student[insert_key] = insert_value
except (ValueError, EOFError):
pass
print(f"After Insertion: {student}")
# 3. Update Operation
try:
update_key = int(input())
update_value = input()
if update_key in student:
student[update_key] = update_value
except (ValueError, EOFError):
pass
print(f"After Update: {student}")
# 4. Deletion Operation
try:
delete_key = int(input())
if delete_key in student:
del student[delete_key]
except (ValueError, EOFError):
```

```
pass
print(f"After Deletion: {student}")
# 5. Traversal Operation
print("Traversing Dictionary:")
for key, value in student.items():
print(f"{key} : {value}")
if _name _ = " _main _":
main()
```

07:13

mitaoe.codetantra.com/

CODETANTRA

```
    2: "Riya",
    3: "Kiran",
    4: "Neha",
    5: "Arjun",
    6: "Pooja",
    7: "Rahul",
    8: "Sneha",
    9: "Vikram",
    10: "Anjali"
}
print("Original Dictionary:",student)
key1=int(input())
```

Test case 1

91 ms

Expected output	Actual output
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahu	Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooj

07:13

mitaoe.codetantra.com/

CODETANTRA

```
    2: "Riya",
    3: "Kiran",
    4: "Neha",
    5: "Arjun",
    6: "Pooja",
    7: "Rahul",
    8: "Sneha",
    9: "Vikram",
    10: "Anjali"
}
print("Original Dictionary:",student)
key1=int(input())
```

Suresh

```
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'
```

Suresh

```
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'An
```

07:13

mitaoe.codetantra.com/

CODETANTRA

```
    2: "Riya",
    3: "Kiran",
    4: "Neha",
    5: "Arjun",
    6: "Pooja",
    7: "Rahul",
    8: "Sneha",
    9: "Vikram",
    10: "Anjali"
}
print("Original Dictionary:",student)
key1=int(input())
```

Karthik

```
After Update: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Sures
```

Karthik

```
After Update: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Sures
```

07:13

mitaoe.codetantra.com/

CODETANTRA

```
    2: "Riya",
    3: "Kiran",
    4: "Neha",
    5: "Arjun",
    6: "Pooja",
    7: "Rahul",
    8: "Sneha",
    9: "Vikram",
    10: "Anjali"
}
print("Original Dictionary:",student)
key1=int(input())
```

```
After Deletion: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
```

```
After Deletion: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
```

07:13

mitaoe.codetantra.com/

CODETANTRA

```
    2: "Riya",
    3: "Kiran",
    4: "Neha",
    5: "Arjun",
    6: "Pooja",
    7: "Rahul",
    8: "Sneha",
    9: "Vikram",
    10: "Anjali"
}
print("Original Dictionary:",student)
key1=int(input())
```

1 : Amit	1 : Amit
2 : Riya	2 : Riya
3 : Karthik	3 : Karthik
4 : Neha	4 : Neha
6 : Pooja	6 : Pooja
7 : Rahul	7 : Rahul
8 : Sneha	8 : Sneha

2.2. Lab Assignments

2.2.1. Linear Search Technique

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

● The first line of input contains the array of integers which are separated by space

● The last line of input contains the key element to be searched

Output format:

● If the element is found, print the index.

● If the element is not found, print Not found.

Sample Test Case:

Input:

1 2 3 4 3 5 6

3

Output:

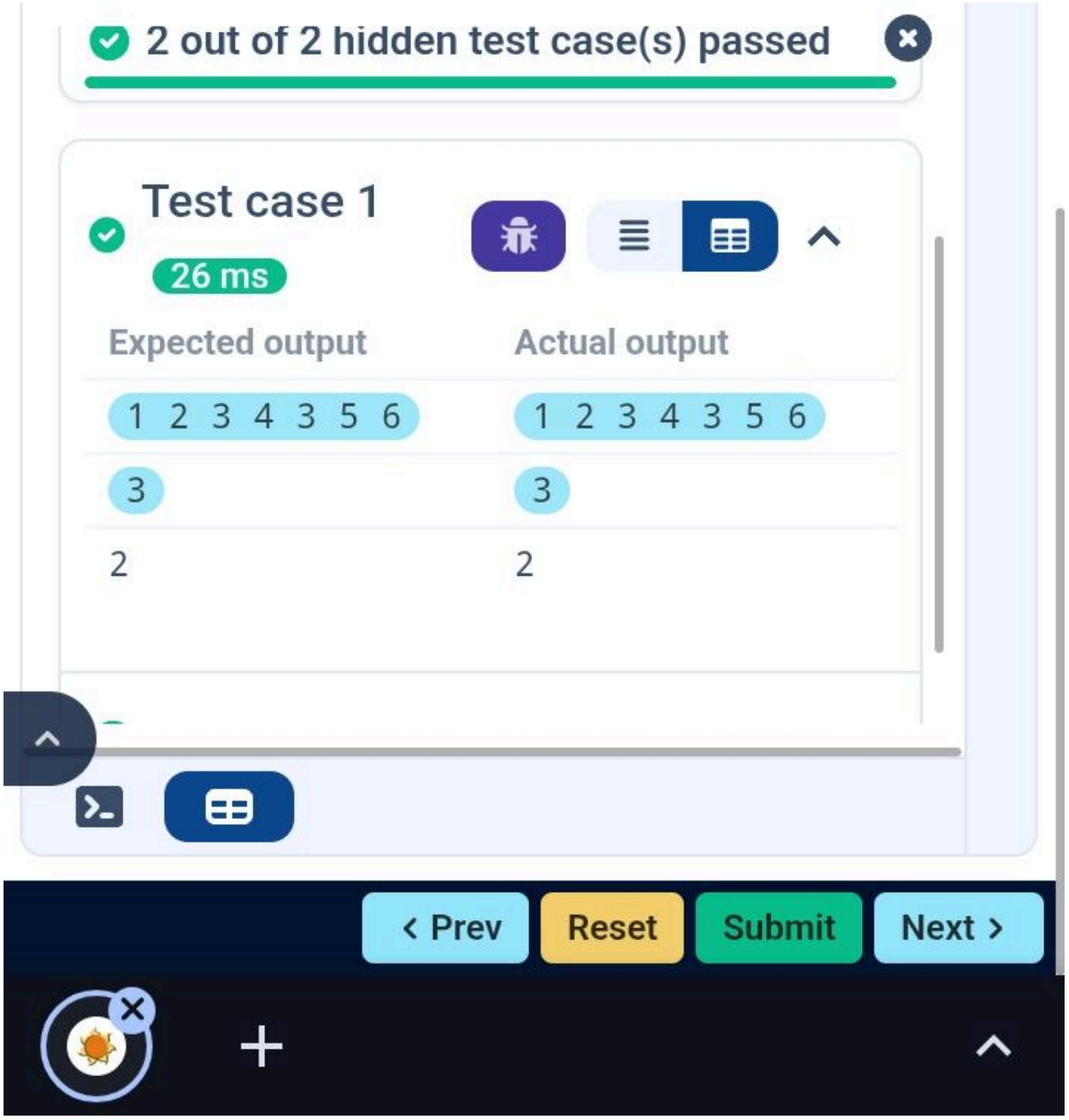
2

```
def main():
# Read the input line, split by spaces and convert each part into an
integer
numbers = list(map(int, input().split()))
# Read the key element to be searched
key = int(input())
# Linear Search Logic
found = False
for i in range(len(numbers)):
if numbers[i] = key:
print(i) # Print the index of the first occurrence
found = True # Mark as found
break # Stop searching once found
if not found:
print("Not found")
if _name _ = " _main _":
main()
```

Aliter: The most Pythonic Way —

```
numbers = list(map(int, input().split()))
key = int(input())
if key in numbers:
print(numbers.index(key))
else:
```

print("Not found")



2.2.2. Captain of the Team

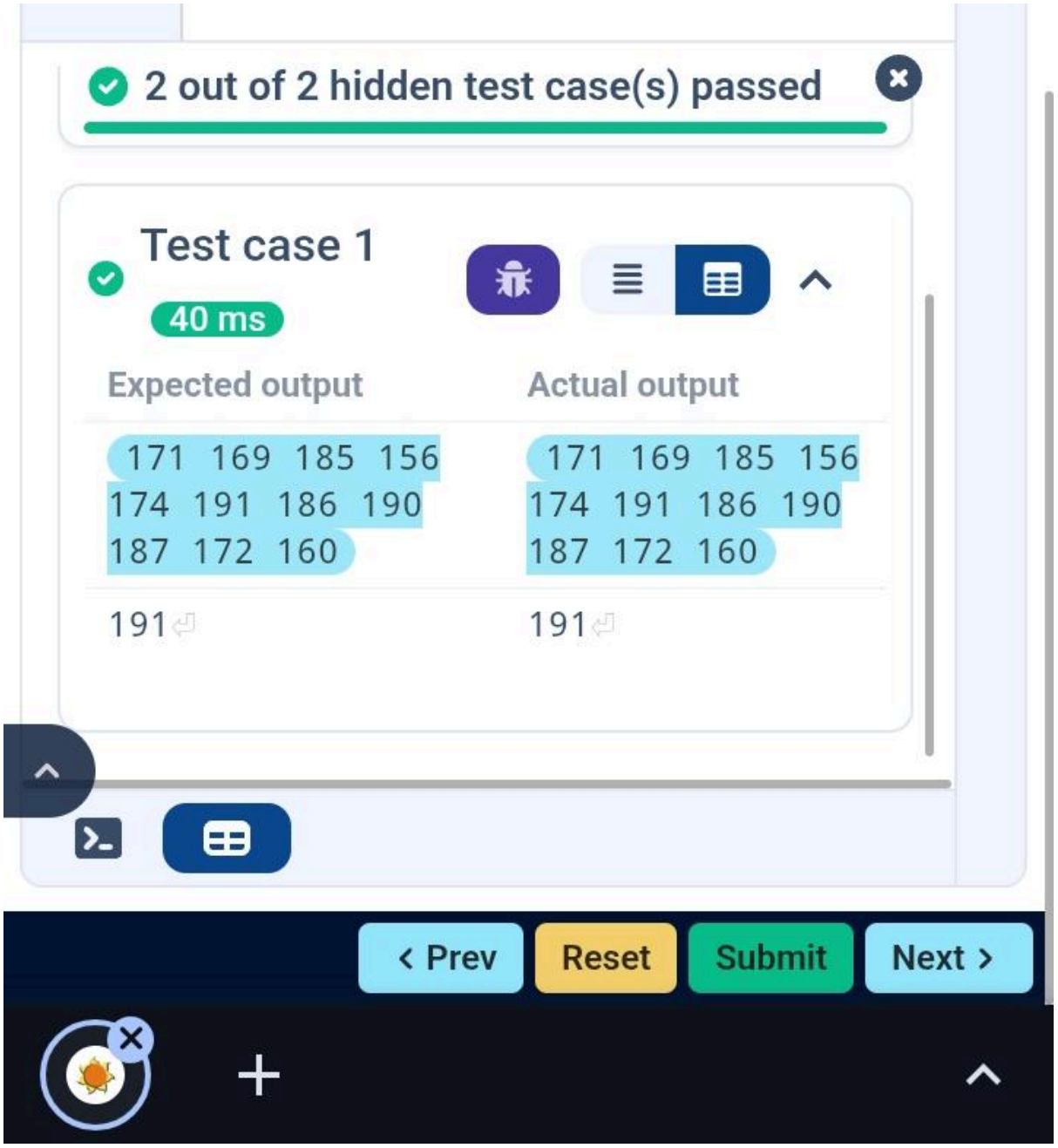
You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format

The output should be the height (in centimeters) of the tallest player.

```
# Read the input line, split by spaces, and convert each part to an
integer
heights = list(map(int, input().split()))
# Find the maximum value in the list
tallest_player = max(heights)
```

```
# Print the result
print(tallest_player)
```